An Intelligent Hint Generation Agent for Competitive Programming

Team Leader
2025040161

Team Member 1
2025040141

TL;DR

We propose an LLM-based agentic workflow to automatically generate progressive, Codeforces-style hints for competitive programming problems, culminating in an interactive educational live demo.

1 Introduction

Competitive Programming (CP), including elite competitions like the Olympiad in Informatics (OI), presents a steep learning curve. Students widely practice on platforms like Codeforces (CF) [1]. However, when stuck, reading full editorial solutions often spoils the problem and bypasses the critical thinking process. A highly effective pedagogical alternative is providing progressive hints (e.g., Hint 1, Hint 2) to guide learners step-by-step.

While current Large Language Models (LLMs) possess strong zero-shot coding capabilities [2], they typically act as direct "solution printers" rather than interactive tutors. Furthermore, simply applying Supervised Fine-Tuning (SFT) rarely improves a model's underlying logic limitation. Therefore, we propose an intelligent agentic workflow. Our system digests a CP problem and its full solution, strategically decomposing the logic to generate high-quality, CF-style progressive hints.

2 Goal of Achievement

The primary goal of this project is to build an interactive AI tutoring system for competitive programming. Specifically, we aim to achieve:

1. **High-Quality Hint Generation:** The system should generate structured, non-spoiling, and logically sound hints mirroring the classic CF tutorial style.
2. **Live Interactive Demo:** We will deliver a web-based UI where a user can input a competitive programming problem and its solution. The system will dynamically generate and reveal hints step-by-step as the user requests them, serving as a real-time coding tutor.

3 Methods

Based on the capabilities of modern LLMs, we will adopt a workflow-driven approach over a pure Supervised Fine-Tuning (SFT) paradigm, structured as follows:

1. Data Collection We will crawl problems, full solutions, and existing human-written progressive hints from Codeforces. This dataset will serve as both a reference for prompt engineering and a training set if style-alignment is required.

2. Agentic Workflow Construction We will utilize a strong base model (e.g., DeepSeek-Coder or an open-source equivalent) as the foundation. We will design a *Multi-Agent* or *Chain-of-Thought (CoT)* workflow [3]:

Preprint. Under review.

- **Decomposer Agent:** Analyzes the problem and the full solution, breaking the logic down into multiple distinct conceptual steps.
- **Reviewer/Critic Agent:** Evaluates the generated hints to ensure they are educational, do not reveal the entire solution at once, and provide progressive guidance.

3. Style Alignment via SFT (Optional/Enhancement) As modern LLMs possess strong 0-shot logic capabilities, SFT will be strictly utilized to align the output *format* and *tone* with the authentic Codeforces tutorial style (if the agentic prompt engineering falls short in style mimicry). We plan to use LoRA for parameter-efficient fine-tuning [4].

4 Timeline

The project will run from May 3, 2026, to the final presentation on June 8, 2026.

- **May 03 - May 10:** Develop crawler; collect and clean Codeforces dataset. Test baseline LLM API zero-shot performance.
- **May 11 - May 20:** Design and implement the Multi-Agent workflow (Decomposer and Critic). Test and evaluate hint quality.
- **May 21 - May 28:** (If necessary) Perform SFT using LoRA to format the output style. Begin backend integration.
- **May 29 - June 04:** Develop the interactive web UI (e.g., using Gradio/Streamlit) for the live demo. Complete system integration.
- **June 05 - June 08:** Final testing, bug fixing, and preparation of the poster for the final presentation.

5 Workload Distribution

Module 1: Data Preparation and Processing (2025040161, more familiar with CP, CF and solution hints with CF style)

Data is the foundation of large language models. Since we aim to imitate the Codeforces style, high-quality training data is extremely important.

Task 1.1: Developing a Codeforces Crawler

- **Goal:** Crawl problem statements and official solutions, namely editorials or tutorials, from Codeforces.
- **Difficulty:**
  - Understanding the anti-crawling mechanisms of CF and extracting texts with specific formats, such as spoiler panels.
  - CF solutions to problems from different periods have different encodings.

Task 1.2: Data Cleaning and Filtering

- **Goal:** Problems with no solutions or solutions of extremely poor quality will be removed. The core criterion is to select solutions that already contain a progressive structure such as "Hint 1, Hint 2, Solution", so that they can serve as high-quality reference materials.
- **Difficulty:** Preserving and cleaning Markdown / LaTeX mathematical formulas.

Task 1.3: Dataset Construction

- **Goal:** Organize the cleaned data into a specific JSON/JSONL format. For example:

```
{
  "problem": "...",
  "full_solution": "...",
```

```
  "hints": ["hint1", "hint2", "hint3"]
}
```

This dataset will be used for subsequent testing comparisons and, optionally, SFT training.

Module 2: Agent Workflow and Prompting

Task 2.1: Baseline Testing (2025040141)

- **Goal:** Select a baseline model, such as GPT-4o, Claude 3.5, or DeepSeek-Coder. Write a basic prompt, such as "Please generate hints based on the problem statement and solution," and observe the output generated directly in a zero-shot setting. Analyze failure cases, such as excessive verbosity, too much irrelevant text, or lack of a Codeforces-style tone.

Task 2.2: Agent Architecture Design and Prompt Writing (2025040141&2025040161)

- **Goal:** Design a multi-step generation pipeline.
  - **Decomposer:** Ask the large language model not to write hints directly at first, but instead to output the key logical chains in the solution.
  - **Generator:** Generate Hint 1, Hint 2, and Hint 3 step by step according to the logical chains.
  - **Critic:** Ask the large language model to review the generated hints and determine whether they are too explicit. If they reveal too much, the model should be asked to rewrite them.

Task 2.3: Implementing the Agent Logic (2025040141&2025040161)

- **Goal:** Use Python to connect the above logic together. This can be implemented by combining plain handwritten code with API calls, or by using a lightweight framework.

Module 3: Model Fine-tuning with SFT (2025040161)

If after completing Module 2, the hints generated by the Agent are logically good but the language style and formatting do not resemble authentic Codeforces editorials, then this module can be started.

Task 3.1: Constructing the SFT Dataset

- **Goal:** Convert the high-quality Codeforces data with hints selected from Module 1 into the instruction-tuning format required for SFT training.

Task 3.2: Model Training with LoRA

- **Goal:** Use an open-source model, such as Qwen-2-Coder, and perform efficient fine-tuning based on LoRA, so that the model is better aligned with the output style of Codeforces.

Task 3.3: Result Comparison and Evaluation

- **Goal:** Compare the performance difference between `Base Model + Agent` and `SFT Model + Agent`.

Module 4: Front-end Interaction and Live Demo Engineering (2025040141)

This part is demo-oriented. An impressive interactive interface directly determines the final score to a large extent.

Task 4.1: Choosing and Building a UI Framework

- **Goal:** It is recommended to use Streamlit or Gradio, as they are very suitable for quickly building interactive interfaces for AI applications.

Task 4.2: Implementing the Interaction Logic

- **Goal:** Provide an input box on the interface, where users can enter a Codeforces problem link or paste the problem text. After the user clicks generate, do not display all hints at once.
- Implement a Codeforces-like spoiler or accordion panel. The user clicks "Show Hint 1" to reveal the first hint, then clicks "Show Hint 2" after reading it to reveal the second hint.

Task 4.3: LaTeX and Mathematical Formula Rendering

- **Goal:** Algorithm problems often contain many mathematical formulas. Use MathJax to ensure that the front end can correctly render mathematical formulas enclosed by `$` or `$$`. Otherwise, the user experience will be poor.

Module 5: Project Deliverables and Defense Preparation

Task 5.1: Writing the Project Proposal (2025040161)

- **Goal:** Clarify the division of labor and submit the proposal on time. The file should be named `project.tex`.

Task 5.2: Human Evaluation with Case Studies (2025040161)

- **Goal:** Select 5–10 classic algorithm problems, run your system to generate hints, and record the results for presentation on the poster. Demonstrate the difference between "the rough baseline performance without the Agent" and "the progressively guided hints generated by your system."

Task 5.3: Academic Poster Design (2025040141)

- **Goal:** Design a poster for the presentation on June 8. The poster should include the problem motivation, the Agent Workflow architecture diagram, which is very important and should be drawn clearly and prominently, comparison results, and screenshots of the demo.

Task 5.4: Demo Rehearsal and Contingency Preparation (2025040141)

- **Goal:** The network or API may be unstable during the defense. Therefore, several high-quality selected cases must be cached or hardcoded in advance in the demo library. During the live presentation, these cases should be demonstrated first.

References

[1] Codeforces. Codeforces - competitive programming platform. `https://codeforces.com/`, 2026.

[2] Tianyu Zheng, Ge Zhang, Tianyu Shen, Xuanjing Liu, Bill Yuchen Lin, Jie Fu, Wenhu Chen, et al. Opencodeinterpreter: Integrating code generation with execution and refinement. *arXiv preprint arXiv:2402.14658*, 2024.

[3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[4] Edward J Hu, Yelong Shen, Phil Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.